

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

...with a Leakage-Aware Evaluation Pipeline

Thai Nguyen Vu¹ Bui Van Dung² Tri Nhut Do² Van Du Nguyen³

FAIR'2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML
...with a Leakage-Aware Evaluation Pipeline

Thai Nguyen Vu¹ Bui Van Dung² Tri Nhut Do² Van Du Nguyen³
FAIR'2026

¹Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

²University of Information Technology, VNU-HCM, Ho Chi Minh City, Vietnam

³Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

Seven minutes on a single claim: on CICIDS2017, a near-perfect F1 is usually a property of the evaluation pipeline, not of the model.

2026-10-02

Dimensionality Reduction and Feature Selection for Network
Intrusion Detection on Apache Spark ML

└ I. PROBLEM AND CONTRIBUTIONS

I. PROBLEM AND
CONTRIBUTIONS

I. PROBLEM AND CONTRIBUTIONS

A near-perfect F1 on CICIDS2017 is a warning sign

Two problems, usually treated separately

- **Cost.** $\approx$ 80 features per flow.
- **Trust.** Near-perfect binary F1 usually signals **data leakage**.

Two concrete leakage sources

1. `destination_port` encodes the *attacked service*.
2. **Feature-level duplicates** survive full-row dedup, so the same sample lands in train *and* test.

This paper

Compare four reduction strategies over the same eight Spark ML classifiers, but first make the *evaluation pipeline* trustworthy.

The contribution is the protocol

Not “which method wins” but **how to measure so the answer means something**.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

I. PROBLEM AND CONTRIBUTIONS

A near-perfect F1 on CICIDS2017 is a warning sign

Frame the talk as a measurement paper. The port gives the model a port-to-label shortcut, and 80 features are expensive to train, to score and to fit on an edge device. Both leakage sources are removed before any split.

A near-perfect F1 on CICIDS2017 is a warning sign		
Two problems, usually treated separately ■ Cost. $\approx$80 features per flow. ■ Trust. Near-perfect binary F1 usually signals data leakage.		This paper Compare four reduction strategies over the same eight Spark ML classifiers, but first make the evaluation pipeline trustworthy.
Two concrete leakage sources 1. <code>destination_port</code> encodes the attacked service. 2. Feature-level duplicates survive full-row dedup, so the same sample lands in train and test.		The contribution is the protocol Not “which method wins” but how to measure so the answer means something .

1. A **uniform comparison** of RF Importance, SHAP and PCA across the same 8 Spark ML algorithms + 2 ensembles on CICIDS2017.
2. SHAP contrasted **directly** against RF Importance on the same training set.
3. A **leakage-aware, statistically tested** pipeline: leak removal, deterministic feature-level dedup, validation-based selection, paired resampling with an *equivalence* test (TOST).
4. A **per-attack-type** and a **cross-dataset** evaluation, which expose what in-domain binary F1 hides.
5. An **edge-aware** recommendation: F1 *and* feature count *and* model size, for a Jetson Orin Nano Super (deployment reported in a companion systems paper).

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

I. PROBLEM AND CONTRIBUTIONS

Contributions

Five contributions. Three, four and five are the ones rarely done together.

Contributions
1. A uniform comparison of RF Importance, SHAP and PCA across the same 8 Spark ML algorithms + 2 ensembles on CICIDS2017.
2. SHAP contrasted directly against RF Importance on the same training set.
3. A leakage-aware, statistically tested pipeline: leak removal, deterministic feature-level dedup, validation-based selection, paired resampling with an equivalence test (TOST).
4. A per-attack-type and a cross-dataset evaluation, which expose what in-domain binary F1 hides.
5. An edge-aware recommendation: F1 and feature count and model size, for a Jetson Orin Nano Super (deployment reported in a companion systems paper).

2026-10-02

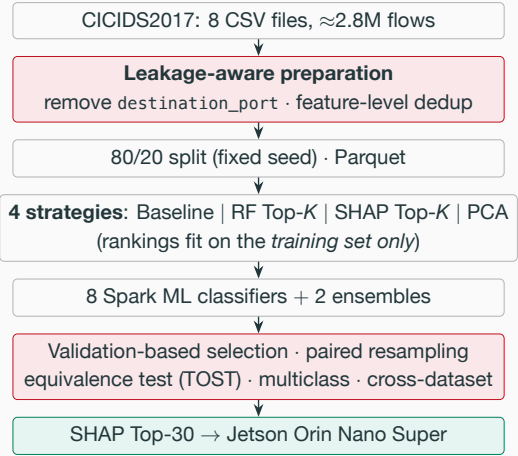
Dimensionality Reduction and Feature Selection for Network
Intrusion Detection on Apache Spark ML

└ II. LEAKAGE-AWARE PIPELINE

II. LEAKAGE-AWARE PIPELINE

II. LEAKAGE-AWARE PIPELINE

The leakage-aware pipeline



Removals happen before the split

- Feature-identical groups with **conflicting labels** are dropped *entirely*: **270 groups, 44,260 rows**.
- Result: 77 features, 1.79M / 0.45M train/test flows.

Design fixed a priori

- Exploration (K sweep) separated from confirmation.
- Hyper-parameters shared across all methods.
- The test set enters **no** selection step.

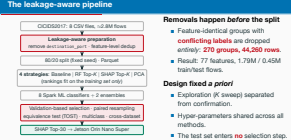
2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

II. LEAKAGE-AWARE PIPELINE

The leakage-aware pipeline

Deterministic deduplication is the part people get wrong. Conflicting groups are removed wholesale, so the result does not depend on how Spark partitions the data.



III. RESULTS AND CONCLUSION

Result 1: 60% of the features are removable

Method	Model	F1	Train (s)	Feats
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0.9970	1226.1	30
PCA k=40	XGBoost	0.9939	1444.8	40
RF Top-30	XGBoost	0.9922	1295.1	30

One run on the original train/test split. Train (s): Spark cluster of 2 x Jetson Orin Nano Super, host as master only.

F1 by algorithm	RF Top-30	SHAP Top-30	ΔF1
Random Forest	0.9879	0.9955	+0.008
XGBoost	0.9922	0.9970	+0.005
GBT	0.9844	0.9918	+0.007
Decision Tree	0.9835	0.9943	+0.011

- SHAP Top-30 keeps baseline F1 while cutting **≈60% of features** and **≈73% of training time**.
- At *equal* feature count, SHAP beats RF Importance on every tree-based algorithm.
- PCA uses *k*=40 because it *extracts* rather than *selects*: each component carries only part of the variance. Cheaper than the baseline, but not explainable.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 1: 60% of the features are removable

Two tables, one message: SHAP Top-30 is the operating point. Same accuracy, 60 percent fewer features, and it is explainable, which matters for the SOC analyst and for the edge board downstream.

Result 1: 60% of the features are removable

Method	Model	F1	Train (s)	Feats
Baseline	XGBoost	0.9971	4615.5	77
SHAP Top-30	XGBoost	0.9970	1226.1	30
PCA k=40	XGBoost	0.9939	1444.8	40
RF Top-30	XGBoost	0.9922	1295.1	30

One run on the original train/test split. Train (s): Spark cluster of 2 x Jetson Orin Nano Super, host as master only.

F1 by algorithm	RF Top-30	SHAP Top-30	ΔF1
Random Forest	0.9879	0.9955	+0.008
XGBoost	0.9922	0.9970	+0.005
GBT	0.9844	0.9918	+0.007
Decision Tree	0.9835	0.9943	+0.011

- SHAP Top-30 keeps baseline F1 while cutting **≈60% of features** and **≈73% of training time**.
- At *equal* feature count, SHAP beats RF Importance on every tree-based algorithm.
- PCA uses *k*=40 because it *extracts* rather than *selects*: each component carries only part of the variance. Cheaper than the baseline, but not explainable.

Result 2: equivalence is tested, not asserted

XGBoost	F1, mean of 6 splits
Baseline (77 features)	0.99734
SHAP Top-30	0.99720
Gap	0.0001

Keeping the port	F1 inflation
4 tree models	+0.0007 to +0.0017
2 linear models	+0.0395 to +0.0415

Baseline and SHAP Top-30 are **equivalent**, so explainability and cost decide. How much the port inflates F1 depends on the model class.

- $N=6$ paired *resamplings*: both methods see the same split each round.
- A significance test cannot show two methods are the same, so we test **equivalence** directly (TOST).
- Result: the true gap is **below 0.00023 F1** at $\alpha = 0.05$.
- The port leak was measured on **all eight** classifiers, not one.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 2: equivalence is tested, not asserted

Say the gap first: one ten-thousandth of F1 over six paired splits. Then the point: we do not argue from a non-significant p-value, we test equivalence. If asked: exact paired permutation over all 2^6 sign flips gives $p = 0.031$, the floor of that test, Cohen's $d \approx 1.94$. TOST against a ± 0.001 margin fixed in advance, on the Nadeau–Bengio error, gives $p = 3.9 \times 10^{-6}$, and the tightest supported margin is $\pm 2.3 \times 10^{-4}$. Port table: per-model rows are in the paper. MLP is +0.0052 and Naive Bayes +0.0108. All later results have the port removed.

Result 2: equivalence is tested, not asserted

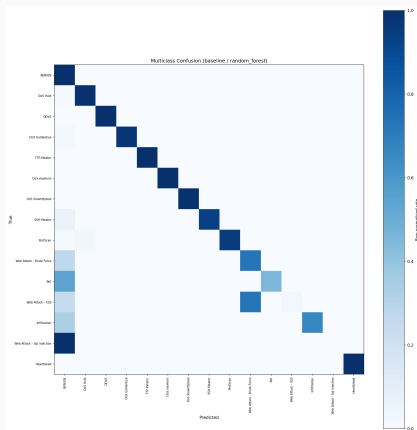
XGBoost	F1, mean of 6 splits
Baseline (77 features)	0.99734
SHAP Top-30	0.99720
Gap	~0.0001

Keeping the port	F1 inflation
4 tree models	+0.0007 to +0.0017
2 linear models	+0.0395 to +0.0415

Baseline and SHAP Top-30 are **equivalent**, so explainability and cost decide. How much the port inflates F1 depends on the model class.

- $N=6$ paired resamplings: both methods see the same split each round.
- A significance test cannot show two methods are the same, so we test **equivalence** directly (TOST).
- Result: the true gap is **below 0.00023 F1** at $\alpha = 0.05$.
- The port leak was measured on **all eight** classifiers, not one.

Result 3: binary F1 hides the rare classes



Class	Support	Recall	F1
Random Forest, all 77 features			
Benign	380,119	0.9997	0.9989
DoS Hulk	34,446	0.9904	0.9947
Web – Brute Force	311	0.7299	0.7323
Bot	290	0.4552	0.6256
Web – XSS	111	0.0270	0.0526
Web – SQL Injection	6	0.0000	0.0000
weighted-F1 = 0.998		macro-F1 = 0.806	

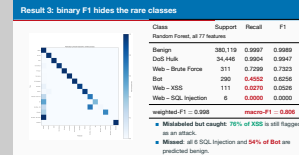
- **Mislabeled but caught:** 76% of XSS is still flagged as an attack.
- **Missed:** all 6 SQL Injection and 54% of Bot are predicted benign.

2026-10-02

III. RESULTS AND CONCLUSION

- Result 3: binary F1 hides the rare classes

Random Forest on all 77 features. Weighted F1 0.998 versus macro F1 0.806. XSS: 81 of 111 land on Web Brute Force, so they are still flagged at the binary level. 27 percent of Brute Force also goes to Benign. Three plausible causes. The imbalance is extreme. With the port removed there is no port-80 shortcut, so short HTTP attack flows look like browsing. And Bot C&C beaconing resembles background traffic. Comparing the reduction methods in multiclass mode is future work.



Result 4: the scores do not survive a different testbed

Train → Test	F1	AUC-PR
2017 → 2017	0.9953	0.9997
2017 → 2018	0.03–0.30	0.4849
2018 → 2018	0.9246	0.9679
2018 → 2017	0.0544	0.1926

18 runs of identical code and data. AUC-PR is their mean.

- Cross-dataset AUC-PR drops **1.00→0.48** and **0.97→0.19**.
- Recall 0.07 / 0.03: the model **misses** the other testbed's attacks.
- Same code, same data, **two F1 bands**: 11 runs at 0.03–0.08, 7 at 0.19–0.30, none between.
- Label-free adaptation (scaler refit, CORAL): **F1 0**. Retraining on **1% target labels**: **0.92 / 0.99**.

Near-perfect in-domain scores, *even leakage-free ones*, do not certify deployability under distribution shift.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Result 4: the scores do not survive a different testbed

Do not soften this. In-domain 0.99, cross-dataset AUC-PR 0.48 / 0.19. Recall 0.073 / 0.030 at precision 0.772 / 0.272: it misses attacks rather than alarming indiscriminately. If asked about the two F1 bands: AUC-PR is the same in both, so the ranking does not change between runs, only where the 0.5 threshold falls against a cluster of near-identical flows. A mean (0.13) would sit in the empty gap, so we report both bands and judge the collapse on AUC-PR. Causes of the shift: CICFlowMeter version, network configuration, attack tooling. 1% of the target is about 19k flows. Consistent with prior cross-dataset IDS studies, and the reason domain adaptation is a prerequisite rather than a nice-to-have.

Result 4: the scores do not survive a different testbed

Train → Test	F1	AUC-PR
2017 → 2017	0.9953	0.9997
2017 → 2018	0.03–0.30	0.4849
2018 → 2018	0.9246	0.9679
2018 → 2017	0.0544	0.1926

18 runs of identical code and data. AUC-PR is their mean.

- Cross-dataset AUC-PR drops **1.00→0.48** and **0.97→0.19**.
- Recall 0.07 / 0.03: the model **misses** the other testbed's attacks.
- Same code, same data, **two F1 bands**: 11 runs at 0.03–0.08, 7 at 0.19–0.30, none between.
- Label-free adaptation (scaler refit, CORAL): **F1 0**. Retraining on **1% target labels**: **0.92 / 0.99**.

Near-perfect in-domain scores, even leakage-free ones, do not certify deployability under distribution shift.

Conclusion

What we showed

- SHAP Top-30 retains baseline F1 with **≈60% fewer features**.
- Baseline vs. SHAP Top-30 are **equivalent** within $\pm 2.3 \times 10^{-4}$ F1.
- **macro-F1 0.806** against weighted-F1 0.998.
- Cross-dataset AUC-PR **collapses to 0.48 / 0.19**. Retraining on 1% of target labels recovers it.

Thank you. Questions?

The contribution

- **How to evaluate reliably**, as much as which method wins.

Next

- Full CSE-CIC-IDS2018 and newer sets (RoEduNet, CIC-IoT).
- Does the SHAP Top-30 *set itself* transfer?
- An ablation of the deduplication step.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Conclusion

Close on the protocol, not the leaderboard.

Conclusion

What we showed

- SHAP Top-30 retains baseline F1 with **≈60% fewer features**.
- Baseline vs. SHAP Top-30 are **equivalent** within $\pm 2.3 \times 10^{-4}$ F1.
- **macro-F1 0.806** against weighted-F1 0.998.
- Cross-dataset AUC-PR **collapses to 0.48 / 0.19**. Retraining on 1% of target labels recovers it.

The contribution

- **How to evaluate reliably**, as much as which method wins.

Next

- Full CSE-CIC-IDS2018 and newer sets (RoEduNet, CIC-IoT).
- Does the SHAP Top-30 *set itself* transfer?
- An ablation of the deduplication step.

Thank you. Questions?

Key references

[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. *Toward generating a new intrusion detection dataset and intrusion traffic characterization*. ICISSP 2018. **CICIDS2017**

[2] Lanvin, M. et al. *Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes*. CRiSIS 2022. **label conflicts**

[3] Arp, D. et al. *Dos and Don'ts of Machine Learning in Computer Security*. USENIX Security 2022. **leakage pitfalls**

[4] Lundberg, S. M. & Lee, S.-I. *A unified approach to interpreting model predictions*. NeurIPS 2017. **SHAP**

[5] Nadeau, C. & Bengio, Y. *Inference for the Generalization Error*. Machine Learning 52(3), 2003. **corrected CI**

[6] D'hooge, L. et al. *Inter-dataset generalization strength of supervised machine learning methods for intrusion detection*. J. Information Security and Applications 54, 2020. **cross-dataset**

[7] Zaharia, M. et al. *Spark: Cluster computing with working sets*. HotCloud 2010. **Spark ML**

[8] CIC & CSE. *CSE-CIC-IDS2018 dataset*. 2018. **cross-dataset target**

[9] Schuirmann, D. J. *A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability*. J. Pharmacokinetics and Biopharmaceutics 15(6), 1987. **TOST**

[10] Sun, B., Feng, J. & Saenko, K. *Return of frustratingly easy domain adaptation*. AAAI 2016. **CORAL**

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

III. RESULTS AND CONCLUSION

Key references

Not presented. Shown while taking questions, so sources are on screen if the audience asks. Full list is in the paper.

Key references	
[1] Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. ICISSP 2018.	CICIDS2017
[2] Lanvin, M. et al. Errors in the CICIDS2017 Dataset and the Significant Differences in Detection Performances It Makes. CRiSIS 2022.	label conflicts
[3] Arp, D. et al. Dos and Don'ts of Machine Learning in Computer Security. USENIX Security 2022.	leakage pitfalls
[4] Lundberg, S. M. & Lee, S.-I. A unified approach to interpreting model predictions. NeurIPS 2017.	SHAP
[5] Nadeau, C. & Bengio, Y. Inference for the Generalization Error. Machine Learning 52(3), 2003.	corrected CI
[6] D'hooge, L. et al. Inter-dataset generalization strength of supervised machine learning methods for intrusion detection. J. Information Security and Applications 54, 2020.	cross-dataset
[7] Zaharia, M. et al. Spark: Cluster computing with working sets. HotCloud 2010.	Spark ML
[8] CIC & CSE. CSE-CIC-IDS2018 dataset. 2018.	cross-dataset target
[9] Schuirmann, D. J. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. J. Pharmacokinetics and Biopharmaceutics 15(6), 1987.	TOST
[10] Sun, B., Feng, J. & Saenko, K. Return of frustratingly easy domain adaptation. AAAI 2016.	CORAL

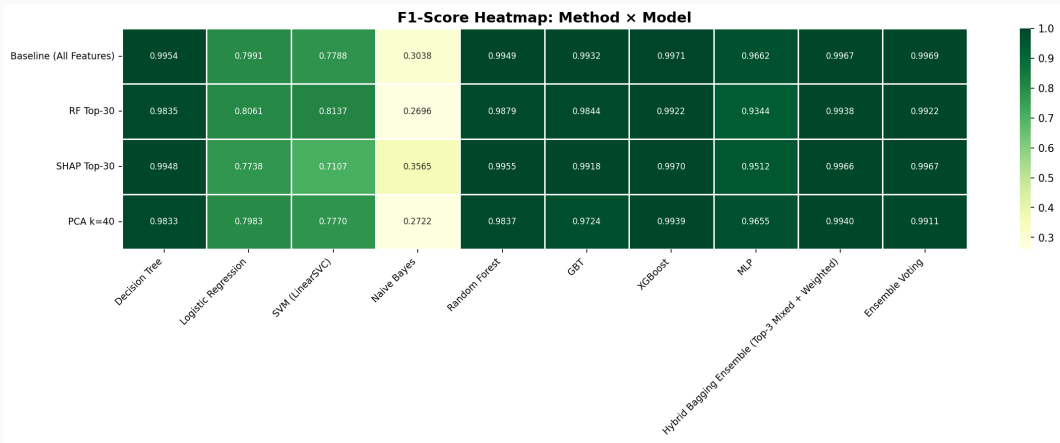
2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup slides

Backup slides

Backup: F1 heatmap, method × algorithm

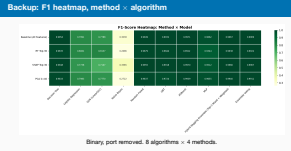


Binary, port removed, 3 algorithms × 4 methods.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: F1 heatmap, method × algorithm



Backup: hyper-parameters, fixed a priori

Model	Main hyper-parameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01

Shared across all reduction methods, so differences reflect the *method*, not per-model tuning.
Class weights for 6 models, scale_pos_weight for XGBoost. MLP is unweighted (Spark ML has no sample weights for it), as stated in the limitations. LightGBM excluded: its SynapseML native library is x86_64-only, and the deployment target is ARM64.

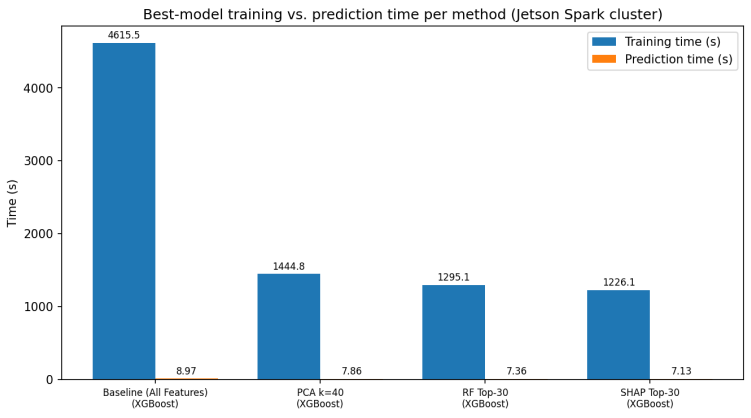
2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: hyper-parameters, fixed a priori

Backup: hyper-parameters, fixed a priori	
Model	Main hyper-parameters
Decision Tree	maxDepth=15, minInstancesPerNode=5, impurity=entropy
Logistic Regression	maxIter=200, regParam=0.001, L2, threshold=0.5
SVM (LinearSVC)	maxIter=200, regParam=0.001
Naive Bayes	gaussian, smoothing=1.0
Random Forest	numTrees=200, maxDepth=15, featureSubset=sqrt
GBT	maxIter=150, maxDepth=6, stepSize=0.05, subsample=0.8
XGBoost	n_estimators=300, max_depth=8, lr=0.05, subsample=0.8, colsample=0.8
MLP	layers=[d,64,32,2], maxIter=150, stepSize=0.01
Shared across all reduction methods, so differences reflect the method, not per-model tuning. Class weights for 6 models, scale_pos_weight for XGBoost. MLP is unweighted (Spark ML has no sample weights for it), as stated in the limitations. LightGBM excluded: its SynapseML native library is x86_64-only, and the deployment target is ARM64.	

Backup: training and prediction cost



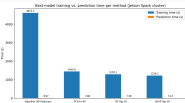
Measured on the Spark training cluster. Latency/throughput *on the edge device* belong to the companion systems paper.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: training and prediction cost

Backup: training and prediction cost



Measured on the Spark training cluster. Latency/throughput on the edge device belong to the companion systems paper.

Backup: threats to validity

- **Internal.** RF/SHAP rankings come from the original training set and stay fixed across resamplings, consistent with the a-priori design, but a possible leak in the ranking step. Re-ranking inside two of the six resamplings moved F1 by $\leq 7.4 \times 10^{-5}$, and 25–26 of 30 features were reselected. MLP carries the unweighted-training caveat.
- **External.** CICIDS2017 may not reflect current traffic, and benchmark NIDS datasets have documented quality limitations. The cross-dataset conclusion rests on a *stratified sample* of CSE-CIC-IDS2018. Evasive traffic untested. Feature-level dedup itself shifts the benign/attack distribution.
- **Statistical.** $N=6$ gives low power: two-sided exact enumeration floors p at $2/2^N$, so $N=6$ just reaches 0.05. The equivalence claim does not rest on that p but on TOST, on the Nadeau–Bengio error that accounts for overlapping resamplings. Cross-dataset F1 is not deterministic across runs, so it is reported over 18 runs.

2026-10-02

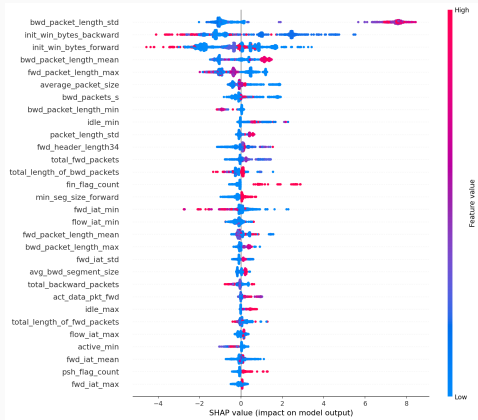
Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: threats to validity

Backup: threats to validity

- **Internal.** RF/SHAP rankings come from the original training set and stay fixed across resamplings, consistent with the a-priori design, but a possible leak in the ranking step. Re-ranking inside two of the six resamplings moved F1 by $\leq 7.4 \times 10^{-5}$, and 25–26 of 30 features were reselected. MLP carries the unweighted-training caveat.
- **External.** CICIDS2017 may not reflect current traffic, and benchmark NIDS datasets have documented quality limitations. The cross-dataset conclusion rests on a *stratified sample* of CSE-CIC-IDS2018. Evasive traffic untested. Feature-level dedup itself shifts the benign/attack distribution.
- **Statistical.** $N=6$ gives low power: two-sided exact enumeration floors p at $2/2^N$, so $N=6$ just reaches 0.05. The equivalence claim does not rest on that p but on TOST, on the Nadeau–Bengio error that accounts for overlapping resamplings. Cross-dataset F1 is not deterministic across runs, so it is reported over 18 runs.

Backup: SHAP feature attribution



SHAP is computed on a *class-stratified* sample so both benign and attack traffic are represented in the ranking.

2026-10-02

Dimensionality Reduction and Feature Selection for Network Intrusion Detection on Apache Spark ML

Backup: SHAP feature attribution

